

Amazon ECS Services: Task Management and Fargate Capacity

1. What Is an ECS Service?

An **Amazon ECS Service** is a long-running manager or scheduler that ensures a specified number of ECS Tasks are running and healthy.

Think of it this way:

- **Task** → The actual running application/container
- **Service** → The supervisor that keeps the required number of Tasks running

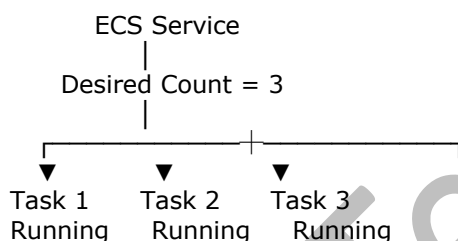
For example, suppose you have a web application and want **3 instances** running at all times.

You configure:

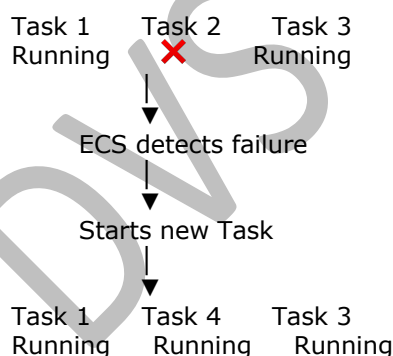
Desired Count = 3

The ECS Service continuously monitors the Tasks and works to maintain that desired count.

If one Task fails, ECS automatically starts a replacement.



If Task 2 crashes:

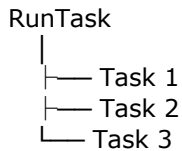


The Service restores the desired count back to **3**.

2. ECS Service vs. Multiple Standalone Tasks

You can run multiple ECS Tasks directly using operations such as RunTask.

For example:



However, these Tasks are **not managed as a group by an ECS Service**.

There is a major difference between manually running multiple Tasks and running those Tasks through an ECS Service.

Feature	Standalone Tasks	ECS Service
Primary Use	One-off jobs, batch processing, migrations, scheduled work	Web servers, APIs, workers, long-running applications
Self-Healing	❌ Failed Task remains stopped	✅ Failed Tasks are replaced
Desired Count	❌ No continuous management	✅ Maintains desired count
Load Balancer	❌ No automatic Service-based integration	✅ Integrates with ALB/NLB
Deployments	Manual	✅ Managed deployments
Auto Scaling	❌ No Service-level scaling	✅ Supports Service Auto Scaling
Long-Running Apps	Not ideal	✅ Designed for this

3. Why Use an ECS Service?

3.1 Self-Healing and Auto-Recovery

One of the most important responsibilities of an ECS Service is **maintaining the desired number of Tasks**.

Without a Service

Suppose you manually start 3 Tasks:



If Task 2 crashes:



You now have only **2 running Tasks**.

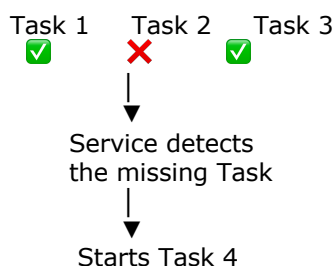
ECS will not automatically maintain the count because there is no Service managing those Tasks.

With an ECS Service

The Service continuously works toward:

Desired Count = 3

If a Task fails:



Result:



This is the foundation of **self-healing** in ECS.

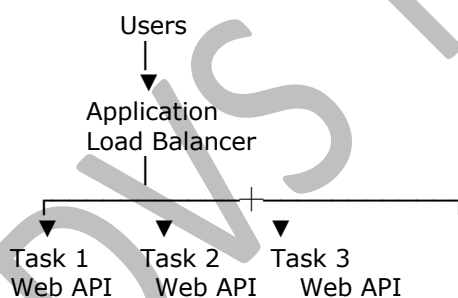
4. Load Balancer Integration

ECS Services can integrate with AWS load balancers such as:

- **Application Load Balancer (ALB)**
- **Network Load Balancer (NLB)**

This is particularly useful for web applications and APIs.

For example:



When the ECS Service launches a new Task, the Task can be registered with the configured target group.

If an old Task is removed, the Service and load-balancing configuration can ensure traffic is directed away from that Task.

This allows the application to dynamically add and remove instances without manually managing the load balancer.

5. Standalone Tasks vs. ECS Service — When to Use What?

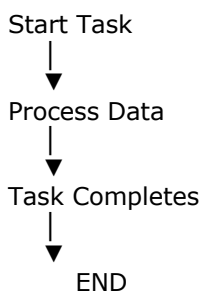
Use Standalone Tasks When:

The workload has a clear beginning and end.

Examples:

- Database migrations
- Batch processing
- One-off scripts
- Data processing jobs
- Ad-hoc administrative tasks
- Scheduled jobs

Example:



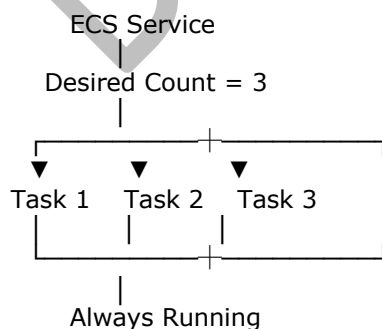
Use an ECS Service When:

The application needs to continuously run.

Examples:

- REST APIs
- Web applications
- Backend services
- Microservices
- Long-running workers
- Production applications

Example:



6. ECS Fargate vs. Fargate Spot

ECS Services can use different **capacity providers**, including:

- FARGATE
- FARGATE_SPOT

These provide different cost and availability characteristics.

Feature	AWS Fargate	AWS Fargate Spot
Pricing	Standard Fargate pricing	Significantly discounted, with savings varying over time
Capacity	Standard Fargate capacity	Spare AWS capacity
Interruption	Not subject to Fargate Spot interruption	Can be interrupted when AWS needs the capacity
Interruption Notice	Not applicable	Typically 2-minute warning
Best For	Production workloads requiring predictable availability	Fault-tolerant, interruption-tolerant workloads
Examples	APIs, web applications, critical services	Batch jobs, background workers, stateless processing

7. Understanding Fargate Spot

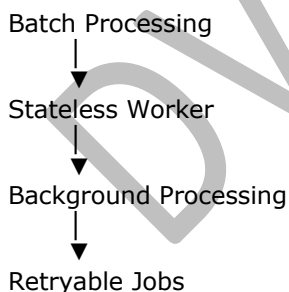
Fargate Spot allows ECS Tasks to run using spare AWS capacity at a lower price.

The trade-off is that AWS may reclaim the capacity.

When this happens, the Task receives a termination warning and is given approximately **2 minutes** before termination.

Therefore, Fargate Spot is best suited for workloads that can tolerate interruption.

Good examples:



It is generally not appropriate to depend on Fargate Spot for a single critical Task where interruption would immediately affect users.

8. Combining Fargate and Fargate Spot

You can configure an ECS Service with a **capacity provider strategy** to distribute Tasks across Fargate and Fargate Spot.

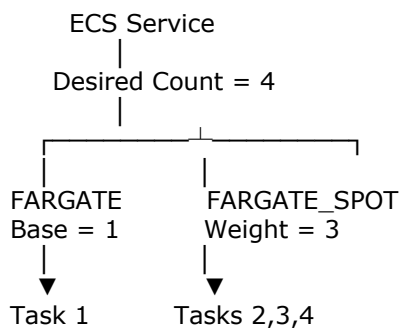
For example:

Capacity Provider	Base	Weight
FARGATE	1	1
FARGATE_SPOT	0	3

This means:

- At least **1 Task** is placed on standard Fargate.
- Additional Tasks are distributed according to the weights.
- Fargate Spot receives a larger share of the additional capacity.

Conceptually:



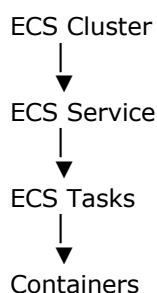
This approach can help balance **reliability and cost**.

ECS Service

- ├─ Maintains desired Task count
- ├─ Replaces failed Tasks
- ├─ Integrates with Load Balancers
- ├─ Manages deployments
- ├─ Supports Auto Scaling
- └─ Can use capacity providers
 - ├─ FARGATE
 - └─ FARGATE_SPOT

9. Key Takeaways

Remember the hierarchy



Simple mental model

Task = Application instance

Service = Manager that keeps application instances running

Fargate = Serverless compute for running ECS Tasks

Fargate Spot = Discounted Fargate capacity that can be interrupted

The most important distinction

If your application needs to **stay running continuously**, use an **ECS Service**.

If you need to **run something once and let it finish**, a **standalone ECS Task** may be more appropriate.

For production services, the typical architecture is:

